

学 士 論 文

題 目 DNA ストレージにおけるナノ
ポアシーケンサーの読み出し特
性を考慮した誤り訂正手法

指導教員 佐藤 高史 教授

京都大学工学部 電気電子工学科

氏 名 岩瀬 真太郎

令和8年2月10日

Error Correction Method Considering the Reading Characteristics of Nanopore Sequencers in DNA Storage

Shintaro Iwase

Abstract

In today’s information society, large amounts of digital data are generated every day, and the amount of data worldwide continues to grow explosively. DNA storage has emerged as a promising solution for long-term archival storage due to its high information density and durability. While nanopore sequencers offer advantages in reading long DNA sequences at high speeds, they suffer from lower accuracy compared to short-read sequencers, particularly with insertion and deletion errors. Although existing error-correcting codes have been proposed for DNA storage with capability to correct these errors, their decoding algorithms have not been optimized for the specific characteristics of nanopore sequencers.

This study aims to optimize existing error correction methods specifically for DNA storage systems using nanopore sequencers. The proposed method incorporates two key improvements: (1) optimization of reward and penalty parameters based on statistical transition matrices that model nanopore reading characteristics, and (2) integration of confidence scores from the basecaller’s CTC (Connectionist Temporal Classification) output into the decoding process. Additionally, to address accuracy degradation with longer DNA sequences, we propose a segmentation-based encoding scheme that divides sequences into short segments with separators and reinitializes the hash function at segment boundaries.

Simulation results demonstrate that the proposed method improves both decoding efficiency and accuracy. The optimized decoding algorithm reduces heap search operations by 64-68% and bit deletion rates by 76-83% compared to the existing method. The segmentation scheme maintains stable decoding accuracy even for DNA sequences up to 15,000 nt without significantly increasing computational complexity.

目次

第 1 章	Introduction	1
第 2 章	Encoding and Readout in DNA Storage	3
2.1	Overview of DNA Storage	3
2.2	HEDGES	4
2.2.1	Encoding	4
2.2.2	Decoding	6
2.3	Nanopore Sequencers and Basecallers	7
2.4	Limitations of Existing Methods and Research Objective	8
第 3 章	Error Correction Method Assuming Readout by Nanopore Sequencers	10
3.1	Improving Search Efficiency in Decoding	10
3.1.1	Modeling Readout Characteristics of Nanopore Sequencers .	10
3.1.2	Treatment of CTC Output	11
3.1.3	Optimization of Rewards and Penalties	12
3.2	Improved Encoding Algorithm for Long Reads	13
第 4 章	Simulation Evaluation	15
4.1	Decoding Search Efficiency Improvement	15
4.1.1	Experimental Method	15
4.1.2	Experimental Conditions	16
4.1.3	Results and Discussion	16
4.2	Improved Encoding Algorithm for Long Reads	17
4.2.1	Experimental Method	17
4.2.2	Experimental Conditions	18
4.2.3	Results and Discussion	18

第 5 章 結論	20
謝 辞	21
参 考 文 献	22
付 録 A 復号の探索効率化における予備実験	25
A.1 実験方法	25
A.2 実験条件	27
A.3 結果	28

第1章 Introduction

In today's information society, massive amounts of digital data are generated every day, and the global volume of data continues to increase explosively. According to a 2025 report, the total volume of digital data worldwide is estimated to be 173.4 zettabytes, and is projected to reach 527.5 zettabytes by 2029[1]. Among this vast amount of data, approximately 60% is known to be cold data that is accessed very infrequently[2]. Cold data includes information that must be retained for long periods to comply with legal requirements and research data that may be analyzed in the future, which makes deletion difficult. Therefore, archival storage technologies that can store cold data for long periods at low cost are strongly needed [3, 2].

In recent years, DNA storage has attracted attention as a next-generation archival storage technology. DNA storage is a technique in which digital data are encoded into sequences of the four nucleotides adenine (A), cytosine (C), guanine (G), and thymine (T), and then stored in synthetically generated DNA. In addition to its high durability, which enables data preservation for thousands of years under appropriate storage conditions, DNA storage is expected to provide extremely high information density because a very large amount of information can be stored per unit volume. For these reasons, it is considered a promising new storage technology suitable for preserving cold data [4, 5, 2].

In DNA storage, data are read by analyzing DNA base sequences using DNA sequencers[5]. DNA sequencers are broadly classified into short-read sequencers, which target DNA with short lengths of several tens to several hundreds of bp * [6], and long-read sequencers, which target DNA with lengths from several hundred to several tens of thousands of bp [7]. However, in many previous studies on DNA storage, short-read sequencers based on SBS (Sequence By Synthesis)[8] have mainly been used. Although SBS provides highly accurate, massively parallel analysis of DNA fragments, it has disadvantages such as a limited read length of only several

* Abbreviation of base pair. A unit representing sequence length in double-stranded DNA. For single-stranded DNA, nucleotides are typically represented by the unit nt.

hundred bp at a time and relatively low sequencing speed[8]. In contrast, nanopore sequencers, one type of long-read sequencer, can analyze very long DNA sequences of up to several hundred thousand bp at high speed, and are therefore considered more practical for DNA storage readout [7]. Accordingly, this study focuses on DNA storage systems that assume readout by nanopore sequencers.

While nanopore sequencers can analyze long DNA sequences rapidly, their base calling accuracy is known to be lower than that of SBS, especially due to the high frequency of insertion and deletion errors [7, 9]. Therefore, when nanopore sequencers are used in DNA storage, it is important to investigate error-correcting codes that can handle insertion and deletion errors in addition to substitution errors.

Previous work proposed error-correcting codes for DNA storage that satisfy sequencing constraints, such as prohibiting consecutive identical bases (homopolymers) and restricting GC content, while correcting substitution, insertion, and deletion errors. Through simulation and experiments using synthesized DNA, the study showed the possibility of accurately recovering petabyte-scale data from base sequences containing up to approximately 10% errors[10]. However, the previous work assumed short-read sequencers for DNA storage readout, and did not optimize the error-correcting code for the readout characteristics of nanopore sequencers. In addition, the previous method reported degraded decoding accuracy for longer base sequences, and improving decoding accuracy for long sequences with long-read sequencers such as nanopore sequencers remains a challenge.

In this study, we propose an error-correction method for DNA storage that assumes readout with nanopore sequencers. We optimize the decoding algorithm of existing error-correction methods by incorporating the readout characteristics of nanopore sequencers to improve both decoding efficiency and accuracy. We also propose a method to suppress accuracy degradation when handling long DNA sequences in the encoding process. We evaluate the proposed method through simulations of the full pipeline, from binary data encoding to nanopore-based readout and decoding.

第2章 Encoding and Readout in DNA Storage

2.1 Overview of DNA Storage

DNA storage is a technology that encodes digital data into DNA base sequences and stores data in synthetically generated DNA as a storage medium. DNA storage performs data writing and readout through the following steps.

1. Encoding

First, binary data are encoded into DNA base sequences. During encoding, sequence constraints are imposed to suppress synthesis errors and maintain structural stability. Such constraints include prohibiting homopolymers and constraining GC content to a certain range [11]. In addition to introducing redundancy for error correction during decoding, the encoded sequences must also be robust against errors generated during sequencing.

2. Storage

Synthetic DNA is generated based on the encoded base sequences. The synthesized DNA is commonly stored in a dry state to increase storage density and stability [12]. Under such conditions, long-term preservation over thousands of years is considered feasible [13].

3. Data readout

To read data from stored DNA, DNA molecules are first extracted, and then their base sequences are analyzed using DNA sequencers such as SBS sequencers and nanopore sequencers, which are described in detail in Section 2.3. Three types of sequencing errors can occur:

- Substitution error: a read base differs from the original base.
- Insertion error: a base not present in the original sequence is read.

- Deletion error: a base present in the original sequence is not read.

The original binary data are reconstructed from sequences containing these errors by applying error correction.

2.2 HEDGES

HEDGES (Hash Encoded, Decoded by Greedy Exhaustive Search) is an error-correction method that satisfies sequence constraints while correcting substitution, insertion, and deletion errors [10].

2.2.1 Encoding

In HEDGES, data are handled in packet units. Each packet consists of 255 fixed-length DNA strands, and the base length of each strand can be freely configured. Encoding is performed in two stages: Reed-Solomon (RS) encoding, which converts the input bitstream into an error-correctable bitstream, and HEDGES encoding, which converts that bitstream into a base sequence with additional redundancy. Figure 2.1 shows an overview of HEDGES encoding. First, binary data are split into packets, and RS encoding is applied to each packet to obtain one RS-encoded bitstream per packet. This RS code is called the outer code and is responsible for final correction of errors that remain after decoding from base sequence back to bitstream. Next, HEDGES encoding, used as the inner code, is applied to the RS-encoded bitstream to generate the base sequence. One key feature of HEDGES is that the coding rate for bit-to-base conversion is variable. By lowering the coding rate, additional redundancy can be introduced to improve error-correction capability.

The specific HEDGES encoding procedure is described below. Assume the message bitstream $\{b_i\}$ is given as follows:

$$b_i, \quad i = 0, 1, 2, \dots, M, \quad b_i \in \{0, 1\} \quad (2.1)$$

To realize an arbitrary coding rate, the number of bits assigned to each base is varied periodically between 0 and 2 bits. Accordingly, the bitstream $\{b_i\}$ is partitioned

into bit groups v_i of length 0 to 2. Examples of v_i for coding rates $r = 0.750$, $r = 0.500$, and $r = 0.333$ are shown below:

$$r = 0.750 : \quad v_0 = b_0b_1, \quad v_1 = b_2, \quad v_2 = b_3b_4, \quad v_3 = b_5, \quad v_4 = b_6b_7, \dots \quad (2.2)$$

$$r = 0.500 : \quad v_0 = b_0, \quad v_1 = b_1, \quad v_2 = b_2, \quad v_3 = b_3, \dots \quad (2.3)$$

$$r = 0.333 : \quad v_0 = b_0, \quad v_1 = b_1, \quad v_2 = 0, \quad v_3 = b_2, \quad v_4 = b_3, \quad v_5 = 0, \dots \quad (2.4)$$

In Eq. (2.2), adjacent bits denote a 2-bit value. For $r = 0.750$, the bit assignment per base alternates between 2 bits and 1 bit. For $r = 0.500$, each base is assigned 1 bit. For $r = 0.333$, the bit assignment pattern repeats as 1 bit, 1 bit, and 0 bit. Let base C_i be assigned to each bit group v_i :

$$C_i, \quad i = 0, 1, 2, \dots, N, \quad C_i \in \{C_i^*\} \quad (2.5)$$

Here, $\{C_i^*\}$ is the candidate set of bases allowed for C_i under sequence constraints, determined by several previously output bases. For example, when a maximum homopolymer length is imposed, if recent output bases already form a maximum-length homopolymer, the same base is excluded from the next output candidates. Similarly, under a GC-content constraint, either A/T or C/G can be excluded based on the recent GC ratio. Therefore, the cardinality of $\{C_i^*\}$, denoted $\#C_i^*$, varies within $0 < \#C_i^* \leq 4$. Assume each base in $\{C_i^*\}$ is assigned an integer value starting from 0. Then the i -th output base C_i is given by:

$$K_i = F(S_i, I_i, V_i) \quad (2.6)$$

$$C_i = K_i + v_i \pmod{\#C_i^*} \quad (2.7)$$

Here, F is a hash function, S_i is a salt based on the DNA strand ID, I_i is the bit-position index, and V_i is the previous 12-bit value. Thus, each output base corresponding to an input bit is determined pseudo-randomly and uniquely by adding the hash value, computed from strand ID, bit position, and recent bit context, to the target bit value and taking the modulo over the number of candidate output bases.

At the beginning of HEDGES encoding, the bit sequence starts with bits representing the DNA strand ID, and in ID encoding $S_i = 0$ is used. Predetermined base sequences called primers are appended to both ends of the encoded base sequence to produce the final DNA strand.

2.2.2 Decoding

To recover the original data from a HEDGES-encoded base sequence, HEDGES decoding is first applied to reconstruct bits from bases while correcting substitution, insertion, and deletion errors. Then RS outer decoding is applied to correct bit sequences that failed to be fully recovered by HEDGES decoding (Fig. 2.1). HEDGES decoding uses a decoding algorithm based on Greedy Exhaustive Search [14]. As illustrated in Fig. 2.2, this algorithm explores a heap structure of candidate bit sequences to select a likely bit sequence for the observed input bases. Each heap element contains a candidate bit sequence and a parameter called skew, which models insertion and deletion hypotheses. Skew indicates how far the referenced base position is shifted from the current position, and is represented as $\Delta \in \{-1, 0, 1\}$. In the decoding process, candidates are first generated for all combinations of bit group v_i and skew Δ , and added to the heap. For each candidate, the predicted base C_i obtained from v_i via Eq. (2.7) is compared with the actually observed base C'_i for that candidate. If they match, a reward is added to the score; otherwise, a penalty is added. At each step, the candidate with the minimum score is selected and expanded as the parent for the next candidates, thereby constructing the heap. Depending on skew, the step penalty ΔP is computed as follows using reward P_{ok} , substitution penalty P_{sub} , insertion penalty P_{ins} , and deletion penalty P_{del} :

- Case $\Delta = 0$:
no positional shift, representing either a match or substitution.

$$\Delta P = \begin{cases} P_{\text{ok}} & (\text{if matched}) \\ P_{\text{sub}} & (\text{if mismatched}) \end{cases} \quad (2.8)$$

- Case $\Delta = -1$:
a deletion is hypothesized, so the input-base reference position is shifted one step backward. In this case, no base comparison is performed.

$$\Delta P = P_{\text{del}} \quad (2.9)$$

- Case $\Delta = 1$:
a one-base insertion is hypothesized, so the input-base reference position is

shifted one step forward.

$$\Delta P = \begin{cases} P_{\text{ins}} + P_{\text{ok}} & \text{(if matched)} \\ P_{\text{ins}} + P_{\text{sub}} & \text{(if mismatched)} \end{cases} \quad (2.10)$$

The score of each hypothesis is accumulated by adding ΔP to the parent hypothesis score. As a result, the correct decoding path tends to have a distinctly lower score than incorrect paths, enabling efficient identification of the correct path. However, for the last few bytes of a message, there are not enough subsequent bases for penalties on wrong hypotheses to accumulate sufficiently, which degrades decoding accuracy. To address this issue, two bytes of zero padding called runout are appended to the end of the message during encoding and removed during decoding.

In heap search during decoding, to prevent explosive growth in computational complexity, a maximum heap size is imposed. When this limit is reached, decoding failure is declared, the search is terminated, and the remaining part of the DNA strand is marked as bit erasures. These bit erasures are finally corrected by the RS outer code.

2.3 Nanopore Sequencers and Basecallers

A nanopore sequencer analyzes DNA base sequences by exploiting the fact that DNA is charged in solution and driving DNA through a protein pore called a nanopore via electrophoresis [7, 9]. Figure 2.3 illustrates the process of DNA analysis by a nanopore sequencer. Multiple nanopore proteins are arranged on the sensor substrate called a flow cell. Each pore has a diameter of about 1 nm, allowing only a single DNA strand to pass through. The flow cell is filled with an electrolyte containing potassium chloride. Because DNA is negatively charged in solution, applying voltage causes electrophoretic motion in the direction of the electric field. This controls DNA movement and guides molecules into nanopores. The guided DNA is unwound from double-stranded to single-stranded DNA by an enzyme called helicase while being regulated to pass through the nanopore at a controlled speed. As DNA passes through the pore, it interferes with ionic flow inside the pore, causing changes in ionic current. Because current changes depend on the sequence of bases

currently passing through the nanopore, DNA base sequences can be determined by analyzing current signals.

The process of determining base sequences from electrical signals is called basecalling, and is performed by machine-learning models called basecallers. Figure 2.4 shows an overview of the Bonito basecaller architecture used in this study [15]. In Bonito, a CNN is first applied to input current signals to extract features. Next, an LSTM captures long-range dependencies in the current signals and extracts temporal features. Then, in the CTC output layer, a linear layer is applied to the LSTM output to produce score sequences over the label set (N, A, C, G, T), where N denotes blank. Beam search is applied to this score sequence to determine the final base sequence. At this stage, the following process, called collapse, is applied to the label sequence:

1. Merge consecutive identical labels into one.
2. Remove blank labels (N).

For example, the label sequence “AANNCCGNGGGTT” is converted to “ACGGT”.

Advantages of nanopore sequencers for DNA storage readout include direct analysis of long DNA sequences, ranging from several hundred bp to several million bp, and high-speed sequencing. On the other hand, controlling DNA translocation speed with helicase is difficult, and basecallers may fail to infer base sequences accurately. Therefore, nanopore sequencers are known to have lower read accuracy than SBS, and error-correcting code design is especially important for DNA storage systems using nanopore sequencers [7, 9].

2.4 Limitations of Existing Methods and Research Objective

The main limitations of the method reported in [10] are as follows:

- Existing studies do not optimize rewards and penalties based on statistical output characteristics when bases are read using nanopore sequencers.

- Because the input to the existing HEDGES decoder is a base sequence, decoding does not utilize the per-time-step probability distribution over bases output by the basecaller.
- In the existing HEDGES encoding/decoding method, the search complexity can explode for long base sequences, making it easier to hit the heap limit and cause decoding failure, which degrades decoding accuracy.

In this study, we propose an error-correction method for DNA storage assuming readout with nanopore sequencers. For rewards and penalties in HEDGES decoding, we optimize them by considering statistical readout characteristics, such as error occurrence probabilities and transition probabilities of each base under nanopore sequencing. By also incorporating base probability distributions output by the basecaller into rewards and penalties, we aim to reduce search complexity and improve decoding accuracy. Furthermore, we propose a method in the encoding process to suppress accuracy degradation when handling long DNA sequences. We evaluate the performance of these proposed methods through simulation.

第3章 Error Correction Method

Assuming Readout by Nanopore Sequencers

3.1 Improving Search Efficiency in Decoding

This section proposes a method to optimize rewards and penalties in decoding by considering both the statistical readout characteristics of nanopore sequencers and the per-time-step base probability distributions in basecaller outputs, in order to improve search efficiency during decoding. First, we describe modeling of nanopore readout characteristics and treatment of CTC outputs from the basecaller. Then, we explain how these are reflected in reward and penalty calculation.

3.1.1 Modeling Readout Characteristics of Nanopore Sequencers

To model readout characteristics of nanopore sequencers, we first construct a transition matrix representing statistical error rates. When both the true base sequence and the post-readout base sequence are known, the two sequences can be compared using sequence alignment to identify matched regions and positions where various errors occur [16]. In aligned sequences, matched base pairs represent correct readout, mismatched pairs represent substitution errors, and bases aligned with gaps represent insertion/deletion errors. From such alignment information, each event in DNA readout, namely correct readout and occurrences of substitution, insertion, and deletion errors, can be represented as transitions among five labels $\{N, A, C, G, T\}$ consisting of four bases plus blank (N).

Using this representation, the statistical readout characteristics of nanopore se-

quencers are described by the following matrix $\mathbf{T}$:

$$\mathbf{T} = \begin{bmatrix} 0 & p_{AN} & p_{CN} & p_{GN} & p_{TN} \\ p_{NA} & p_{AA} & p_{CA} & p_{GA} & p_{TA} \\ p_{NC} & p_{AC} & p_{CC} & p_{GC} & p_{TC} \\ p_{NG} & p_{AG} & p_{CG} & p_{GG} & p_{TG} \\ p_{NT} & p_{AT} & p_{CT} & p_{GT} & p_{TT} \end{bmatrix} \quad (3.1)$$

Each element in this matrix represents the probability of correct readout or occurrence of substitution/insertion/deletion errors when a sufficiently long DNA sequence containing A, C, G, and T in equal proportions is read by a nanopore sequencer. For $X, Y \in \{A, C, G, T\}$:

- p_{XX} : probability that base X is correctly read.
- p_{XY} : probability that base X is substituted by base Y.
- p_{XN} : probability that base X is deleted.
- p_{NY} : probability that base Y is inserted.

The sum of all elements of $\mathbf{T}$ is 1. We define $\mathbf{P}_{\text{STATS}}$ as the matrix obtained by taking the natural logarithm of each element of $\mathbf{T}$ except the first-row/first-column element and multiplying by -1 . This matrix is used later to compute rewards and penalties.

3.1.2 Treatment of CTC Output

As described in Section 2.3, a basecaller usually outputs final base sequences by applying beam search to CTC outputs. In the proposed method, however, the basecaller does not finalize base sequences and instead outputs the CTC score sequence directly. Also, as described in Section 2.2.2, while the existing HEDGES decoder uses base sequences as input, the proposed method uses CTC score sequences as input (Fig. 3.1). This enables incorporation of per-time-step base probability distributions into decoding.

Assume the CTC output is given as an $L \times 5$ matrix $\mathbf{S}$, where L is the time-series length dependent on the input current-signal length, and each row represents

output probabilities over the five labels $\{N, A, C, G, T\}$ at each time step. Applying the following operations, corresponding to collapse in Section 2.3, yields $\mathbf{S}'$:

1. For consecutive rows in $\mathbf{S}$ whose maximum-probability label is the same, average the probabilities for each label and merge them into one row.
2. Remove rows in $\mathbf{S}'$ whose maximum-probability label is blank (N).

Let $\mathbf{P}_{\text{CTC}}$ be the matrix obtained by taking the natural logarithm of each element in $\mathbf{S}'$ and multiplying by -1 . Elements of $\mathbf{P}_{\text{CTC}}$ represent score values based on probabilities that each base or blank is output at each time step. As described in Section 2.2.2, the existing HEDGES decoding algorithm determines rewards and penalties by comparing the base predicted from Eq. (2.7) with the corresponding input base. In the proposed method, rewards and penalties are determined using scores retrieved from $\mathbf{P}_{\text{CTC}}$ based on the output probability of the predicted base for each hypothesis.

3.1.3 Optimization of Rewards and Penalties

In the existing method, reward P_{ok} and penalties P_{sub} , P_{ins} , and P_{del} for substitution, insertion, and deletion errors are fixed values. In this study, we propose new reward/penalty terms P'_{ok} , P'_{sub} , P'_{ins} , and P'_{del} that incorporate nanopore readout characteristics and basecaller CTC outputs described in Sections 3.1.1 and 3.1.2, and define them as follows:

$$P'_{\text{ok}} = P_{\text{ok}} + \alpha_{\text{ok}}(\mathbf{P}_{\text{STATS}}(x_{\text{hypo}}, x_{\text{max}}(i)) - \mu_{\text{STATS}_{\text{ok}}}) + \beta_{\text{ok}}(\mathbf{P}_{\text{CTC}}(i, x_{\text{hypo}}) - \mu_{\text{CTC}_{\text{ok}}}) \quad (3.2)$$

$$P'_{\text{sub}} = P_{\text{sub}} + \alpha_{\text{sub}}(\mathbf{P}_{\text{STATS}}(x_{\text{hypo}}, x_{\text{max}}(i)) - \mu_{\text{STATS}_{\text{sub}}}) + \beta_{\text{sub}}(\mathbf{P}_{\text{CTC}}(i, x_{\text{hypo}}) - \mu_{\text{CTC}_{\text{sub}}}) \quad (3.3)$$

$$P'_{\text{ins}} = P_{\text{ins}} + \alpha_{\text{ins}}(\mathbf{P}_{\text{STATS}}(N, x_{\text{max}}(i-1)) - \mu_{\text{STATS}_{\text{ins}}}) + \beta_{\text{ins}}(\mathbf{P}_{\text{CTC}}(i-1, x_{\text{hypo}}) - \mu_{\text{CTC}_{\text{ins}}}) \quad (3.4)$$

$$P'_{\text{del}} = P_{\text{del}} + \alpha_{\text{del}}(\mathbf{P}_{\text{STATS}}(x_{\text{hypo}}, N) - \mu_{\text{STATS}_{\text{del}}}) \quad (3.5)$$

Here, x_{hypo} denotes the base output from a hypothesis bit sequence, and i denotes the row index of $\mathbf{P}_{\text{CTC}}$ referenced by that hypothesis. $x_{\text{max}}(i)$ denotes the base

with maximum output probability in the i -th row of $\mathbf{P}_{\text{CTC}}$. $\mathbf{P}_{\text{STATS}}(X, Y)$ ($X, Y \in \{N, A, C, G, T\}$) denotes the score corresponding to transition from label X to label Y in $\mathbf{P}_{\text{STATS}}$. $\mathbf{P}_{\text{CTC}}(i, X)$ denotes the score of label X in row i of $\mathbf{P}_{\text{CTC}}$. $\mu_{\text{STATS_ok}}$, $\mu_{\text{STATS_sub}}$, $\mu_{\text{STATS_ins}}$, and $\mu_{\text{STATS_del}}$ denote the expected values of $\mathbf{P}_{\text{STATS}}$ scores when correct readout, substitution, insertion, and deletion errors occur, respectively. $\mu_{\text{CTC_ok}}$, $\mu_{\text{CTC_sub}}$, and $\mu_{\text{CTC_ins}}$ denote the statistical expected values of scores in $\mathbf{P}_{\text{CTC}}$ corresponding to correct readout, substitution error, and insertion error, respectively. Reward and penalties are calculated by adding normalized correction terms, whose expectations are zero, to the fixed values P_{ok} , P_{sub} , P_{ins} , and P_{del} . Each correction term is multiplied by adjustment parameters α_{ok} , α_{sub} , α_{ins} , α_{del} , β_{ok} , β_{sub} , and β_{ins} . The fixed penalties and the weights of correction terms are optimized by simulation. Details of constant settings and parameter optimization are described in the appendix.

3.2 Improved Encoding Algorithm for Long Reads

As described in Section 2.4, in the existing HEDGES encoding/decoding method, decoding accuracy decreases as DNA sequence length increases. This is because when the number of hypotheses reaches the heap-size limit in the decoding algorithm, decoding failure is declared and all subsequent bits are treated as erasures. Therefore, the failure rate increases with sequence length. This section proposes an improved encoding/decoding method that suppresses accuracy degradation when handling long DNA sequences.

In the HEDGES encoder, each output base C_i corresponding to bit group v_i is determined recursively by Eq. (2.7) using a hash function based on DNA strand ID, bit position, and recent bit history. Therefore, for example, if a burst error occurs at some position of a DNA strand, the impact propagates to decoding of subsequent bases, making correct path search difficult. To address this, the proposed method divides long base sequences into 400 nt segments as shown in Fig. 3.2, and initializes the bit position I_i and recent-bit information B_i to zero at the beginning of each segment during encoding. To identify boundaries between adjacent segments, a known separator sequence of about 10 bases with a specific pattern is inserted

between segments. A separator pattern is selected to avoid homopolymers and maintain GC content around 50%. In this study, "GTACTGCATG" is used as the separator.

During decoding, separator positions are first detected by the following procedure:

1. Set $n = l_p + l_{\text{seg}}$, where l_p denotes primer length and l_{seg} denotes segment length.
2. For each integer k satisfying $n - 30 < k < n + 30$, compute $S_{\text{match}}(k)$ by the following equation, and select the k that minimizes $S_{\text{match}}(k)$ as the separator position, denoted k' .

$$S_{\text{match}}(k) = \sum_{j=0}^{l_{\text{sep}}-1} P_{\text{CTC}}(k + j, S_j) \quad (3.6)$$

Here, S_j denotes the j -th base of the separator sequence, and l_{sep} denotes separator length.

3. Set $n = k' + l_{\text{sep}} + l_{\text{seg}}$.
4. Repeat steps 2 and 3 to detect separator positions for all segments.

After separator positions are identified, the decoding algorithm described in Section 2.2.2 is applied to each segment. At the beginning of each segment, I_i and B_i are reset to zero and the heap is reinitialized before decoding. This suppresses decoding failure caused by exceeding heap-size limits, and even if a decoding failure occurs, its impact is confined within a segment so that decoding can resume from the next segment.

第4章 Simulation Evaluation

In this chapter, we evaluate the performance of the proposed methods through simulation. In Section 4.1, we measure the number of search operations and decoding accuracy when using the reward/penalty optimization proposed in Section 3.1, and evaluate its effectiveness by comparison with the existing method. In Section 4.2, we measure decoding accuracy when using the improved encoding/decoding algorithm for long reads proposed in Section 3.2, and evaluate its effectiveness by comparison with the existing method.

4.1 Decoding Search Efficiency Improvement

4.1.1 Experimental Method

Figure 4.1 shows an overview of the experimental procedure used to evaluate, through simulation, the performance of the reward/penalty optimization method proposed in Section 3.1. As a preliminary experiment, we evaluated nanopore readout characteristics by simulation and obtained the matrix $\mathbf{P}_{\text{STATS}}$. Next, in encoding/decoding simulations using random binary data, we optimized the parameters of the correction terms in Eqs. (3.2)–(3.5) using gradient descent, with the number of heap search operations during decoding as the objective function. Parameter optimization was performed independently for multiple coding rates, under three settings: using only correction terms based on nanopore output characteristics, using only correction terms based on CTC output, and using both correction terms. Detailed procedures and conditions of these preliminary experiments are described in the appendix.

Using the optimal parameters obtained in the preliminary experiments, we evaluated decoding performance of the proposed methods. First, binary test data were encoded by HEDGES, and current waveforms for the resulting DNA strands were generated using a simulator called Squigulator[17] for nanopore sequencing. Next,

basecalling was performed on the simulated current waveforms using Bonito. At this stage, we output not only the final base sequence but also the CTC score matrix before CTC decoding. Using these basecalling results, we performed decoding with the existing and proposed methods, and measured the number of heap searches, the number of bit erasures due to decoding failure, and the final bit error rate after RS decoding. In decoding with the existing method, the input is the base sequence output by the basecaller. In decoding with the proposed method, the input is the CTC score matrix, and three cases were evaluated: using only correction terms based on nanopore readout characteristics, using only correction terms based on CTC output, and using both correction terms. This end-to-end simulation from binary-data encoding to decoding was conducted for multiple coding rates.

4.1.2 Experimental Conditions

As test binary data, we used the full text of Kokoro by Natsume Soseki (UTF-8), obtained from Aozora Bunko, with ruby annotations (text enclosed by 《...》) removed using regular expressions[18]. The data size is 481158 bytes. This binary data was encoded by HEDGES at coding rates 0.75, 0.60, 0.50, 0.33, with a base length of 400 nt. For nanopore-current simulation using Squigulator, we used parameter settings assuming a MinION R9.4.1 flow cell[17]. For the Bonito model, we used an existing pretrained model that was fine-tuned for reading DNA encoded by HEDGES[15]. In decoding with both the existing and proposed methods, the maximum heap size was set to 1000000.

4.1.3 Results and Discussion

Table 4.1, Table 4.2, and Table 4.3 show, respectively, the average number of heap searches per DNA strand, the bit-erasure rate due to decoding failure, and the final bit error rate after RS decoding in the decoding performance evaluation of the existing and proposed methods. In these tables, “STATS only” denotes the case where only correction terms based on nanopore readout characteristics are applied, “CTC only” denotes the case where only correction terms based on CTC output are applied, and “STATS+CTC” denotes the case where both correction

terms are applied. By using the proposed method, we reduced the number of heap searches and the decoding-failure rate compared with the existing method at all coding rates. When applying only correction terms based on nanopore readout characteristics, heap searches were reduced by 43% to 61%, and bit-erasure rates were reduced by 60% to 77%. When applying only correction terms based on CTC output, heap searches were reduced by 56% to 65%, and bit-erasure rates were reduced by 68% to 83%. When applying both correction terms, the best performance was obtained: heap searches were reduced by 64% to 68%, and bit-erasure rates were reduced by 76% to 83%. Furthermore, for the final bit error rate after RS decoding at coding rate 0.75, applying both correction terms achieved a 77% error-rate reduction compared with the existing method. These results indicate that the proposed method not only improves decoding search efficiency but also reduces bit erasures due to decoding failure, thereby contributing to improved final decoding accuracy.

4.2 Improved Encoding Algorithm for Long Reads

4.2.1 Experimental Method

To evaluate, through simulation, the performance of the improved encoding/decoding algorithm for long reads proposed in Section 3.2, we compared search complexity and decoding accuracy between the existing method without segmentation and the proposed method with segmentation.

First, we encoded the test binary data using the existing HEDGES encoding algorithm. For the resulting DNA strands, we performed current-waveform simulation and basecalling using the same procedure as in Section 4.1. Using these basecalling results, decoding was performed with the proposed method in Section 3.1, where each parameter was set to the value obtained in Section 4.1. This was repeated while varying the base length per DNA strand from 500 nt to 15000 nt, and the bit error rate in HEDGES decoding was measured. In addition, for encoding with the existing method, we evaluated two cases: one with a fixed heap-size limit regardless of base length, and one where the heap-size limit increases proportionally to base length.

Next, using the same binary data, we encoded with the proposed segmentation method and similarly performed current-waveform simulation and basecalling. In decoding, separator positions must first be detected. Therefore, after computing P_{CTC} from CTC outputs by the method described in Section 3.1.2, we detected separators using the method in Section 3.2, and then decoded each segment using the proposed method in Section 3.1. Keeping segment length fixed and varying the number of segments, we changed the total base length per DNA strand from around 500 nt to around 15000 nt, and measured the bit error rate in HEDGES decoding.

4.2.2 Experimental Conditions

The test binary data were the same as those used in the experiment in Section 4.1. Parameter settings for current simulation by Squigulator and the basecaller model were also the same as in Section 4.1. The coding rate in encoding with both the existing and proposed methods was 0.60. For segmentation in the proposed method, segment length was set to 400 nt. In heap search during decoding for the existing method, two cases were evaluated: one with a fixed heap-size limit of 1000000 regardless of base length, and one where the heap-size limit is increased proportionally to base length with 1000000 at base length 400 nt as the reference point. For segment-wise decoding in the proposed method, the heap-size limit was set to 1000000 for each segment.

4.2.3 Results and Discussion

Figure 4.2 shows the relationship between DNA base length and bit error rate, and Figure 4.3 shows the relationship between base length and average number of heap searches per DNA strand for the existing and proposed methods. Here, Existing method (1) denotes the case where the heap-size limit is fixed regardless of base length, and Existing method (2) denotes the case where the heap-size limit increases proportionally with base length. The experimental results confirmed that in Existing method (1), as reported in [10], bit error rate increases linearly as base length increases. In Existing method (2), increasing the heap-size limit in proportion to base length kept bit error rate approximately constant with respect

to base length, but resulted in the largest number of heap searches among the three methods. In contrast, in the proposed method, bit error rate changed only slightly even as base length increased. Because the upper bound on heap searches in the proposed method is 1000000 per segment, it increased by about 24% compared with Existing method (1), where the whole DNA strand has a limit of 1000000, but was reduced by about 21% compared with Existing method (2).

The degradation in decoding accuracy with the existing method is mainly caused by an increased decoding-failure rate due to growth in heap searches during decoding. In the proposed method, inputs to the hash function and the heap are reinitialized for each segment, which suppresses heap growth and allows decoding to continue from the next segment even if decoding failure occurs, thereby mitigating degradation in decoding accuracy.

第5章 結論

本研究では、DNA ストレージの読み出しにおいてナノポアシーケンサーを用いることを前提とした誤り訂正手法を提案し、その性能をシミュレーションにより評価した。既存手法である HEDGES は、DNA ストレージにおけるシーケンス制約を満たしつつ挿入・消失・置換エラーへの耐性を持つ誤り訂正符号であるが、その復号アルゴリズムにおいて、ナノポアシーケンサーでの DNA 読み出しを前提とした最適化が行われていないという課題と、DNA 鎖の塩基長が長い場合に復号精度が低下するという課題が存在した。

本研究では、これらの課題を解決するために、まず HEDGES の復号アルゴリズムにおいて、ナノポアシーケンサーの統計的な読み出し特性とベースコーラーの CTC 出力を考慮した探索手法を提案した。実験結果より、提案手法では既存手法と比較して復号アルゴリズムにおける探索数を 64% から 68% 削減し、復号失敗によるビット消失率を 76% から 83% 低減できることを示した。さらに、ロングリードに向けた符号化・復号アルゴリズムの改良を提案し、セグメント化による手法を用いることで長い塩基長の DNA を扱う場合の復号精度の低下を抑制できることを示した。

今後の課題としては、以下の点が挙げられる。第一に、ナノポアシーケンサーの出力特性をより詳細にモデル化することである。本研究では、ラベル間の遷移確率のみを考慮した単純なモデルを用いたが、塩基配列のコンテキスト依存性や、バースト的なエラー発生など、より複雑な特性を考慮することで、復号における探索をより最適化できる可能性がある。第二に、ベースコーラーの最新アーキテクチャに対応することである。本研究で使用した Bonito は CNN-LSTM-CTC 構成であるが、最新のベースコーラーでは CRF を用いたアーキテクチャなど、より先進的な機械学習手法が採用されており、これらに対応することで復号精度のさらなる向上が期待できる。第三に、実際の DNA を用いた実験的検証である。本研究はシミュレーションのみによる評価であるため、実際に合成 DNA を用いたナノポアシーケンサーでの読み出し実験を行うことで、提案手法の実用性を検証する必要がある。

謝 辞

本研究を進めるにあたり，ご指導・ご助言を賜りました佐藤高史教授に深く感謝申し上げます。研究会やミーティングを通じて多くの貴重なご指摘・ご助言をいただきました栗野皓光准教授，橋本昌宜教授，上野嶺准教授，白井僚助教，新津葵一教授，劉昆洋助教に深く感謝致します。日々の研究において様々な助言をいただきました小池健文氏に深く感謝いたします。研究室生活において様々な面から支えて頂いた佐藤高史研究室支援職員の西山修子氏，上西香織氏に感謝致します。最後に，日頃から様々なご支援，ご協力を頂きました佐藤研究室，橋本研究室，新津研究室の皆様感謝致します。

参 考 文 献

- [1] Petroc Taylor, Data generation volume worldwide 2010–2029, (2025), <https://www.statista.com/statistics/871513/worldwide-data-created/>, Accessed: 2026-01-26.
- [2] DNA Data Storage Alliance, Preserving our digital legacy: An introduction to dna data storage, Technical report, (DNA Data Storage Alliance, San Diego, CA, 2021).
- [3] David Reinsel, John Gantz, and John Rydning, Data age 2025: The evolution of data to life-critical — don’t focus on big data; focus on the data that’s big, Idc white paper, sponsored by seagate, (International Data Corporation (IDC), Framingham, MA, 2017).
- [4] A. Akash, E. Bencurova, and T. Dandekar, How to make DNA data storage more applicable, *Trends in Biotechnology*, Vol. 42, No. 1, (2024), pp. 17–30.
- [5] Luis Ceze, Jeff Nivala, and Karin Strauss, Molecular digital data storage using DNA, *Nature Reviews Genetics*, Vol. 20, No. 8, (2019), pp. 456–466.
- [6] S. A. Simon, J. Zhai, R. S. Nandety, K. P. McCormick, J. Zeng, D. Mejia, and B. C. Meyers, Short-read sequencing technologies for transcriptional analyses, *Annual Review of Plant Biology*, Vol. 60, (2009), pp. 305–333.
- [7] S. L. Amarasinghe, S. Su, X. Dong, L. Zappia, M. E. Ritchie, and Q. Gouil, Opportunities and challenges in long-read sequencing data analysis, *Genome Biology*, Vol. 21, No. 1, (2020), p. 30.
- [8] M. Uhlen and S. R. Quake, Sequential sequencing by synthesis and the next-generation sequencing revolution, *Trends in Biotechnology*, Vol. 41, No. 12, (2023), pp. 1565–1572, Epub 2023 Jul 21; PMID: 37482467.

- [9] Adam Dorey and Stefan Howorka, Nanopore DNA sequencing technologies and their applications towards single-molecule proteomics, *Nature Chemistry*, Vol. 16, No. 3, (2024), pp. 314–334.
- [10] William H. Press, John A. Hawkins, Stephen K. Jones, Jeffrey M. Schaub, and Ilya J. Finkelstein, Hedges error-correcting code for dna storage corrects indels and allows sequence constraints, *Proceedings of the National Academy of Sciences*, Vol. 117, No. 31, (2020), pp. 18489–18496.
- [11] Joao Gervasio, Henrique Oliveira, André Costa Martins, Joao Pesquero, Bruno Verona, and Natalia Cerize, How close are we to storing data in dna?, *Trends in Biotechnology*, Vol. 42, (2023).
- [12] Dave Landsman and Karin Strauss, The dna data storage model, *IEEE Computer*, Vol. 56, No. 7, (2023), pp. 78–85.
- [13] Robert N. Grass, Reinhard Heckel, Michela Puddu, Daniela Paunescu, and Wendelin J. Stark, Robust chemical preservation of digital information on dna in silica with error-correcting codes, *Angewandte Chemie International Edition*, Vol. 54, No. 8, (2015), pp. 2552–2555.
- [14] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael, A formal basis for the heuristic determination of minimum cost paths, *IEEE Transactions on Systems Science and Cybernetics*, Vol. 4, No. 2, (1968), pp. 100–107.
- [15] Chris Seymour, Oxford Nanopore Technologies Ltd., Bonito: A pytorch base-caller for oxford nanopore reads, (2019), Oxford Nanopore Technologies, Ltd. Public License, v. 1.0.
- [16] Osamu Gotoh, An improved algorithm for matching biological sequences, *Journal of Molecular Biology*, Vol. 162, No. 3, (1982), pp. 705–708.
- [17] Hasindu Gamaarachchi, James M Ferguson, Hiruna Samarakoon, Kisaru Liyanage, and Ira W Deveson, Squigulator: simulation of nanopore sequencing signal data with tunable noise parameters, *bioRxiv*, (2023), pp. 2023–05.
- [18] 夏目漱石, こころ, (1914), <https://www.aozora.gr.jp/cards/000148/card773.html>.

- [19] Peter J. A. Cock, Tiago Antao, Jeffrey T. Chang, Brad A. Chapman, Cymon J. Cox, Andrew Dalke, Iddo Friedberg, Thomas Hamelryck, Frank Kauff, Bartek Wilczynski, and Michiel J. L. de Hoon, Biopython: freely available python tools for computational molecular biology and bioinformatics, *Bioinformatics*, Vol. 25, No. 11, (2009), pp. 1422–1423.
- [20] Clara Delahaye and Jacques Nicolas, Sequencing dna with nanopores: Troubles and biases, *PLOS ONE*, Vol. 16, No. 10, (2021), pp. 1–29.

付 録 A 復号の探索効率化における予備実験

A.1 実験方法

まず、ナノポアシーケンサーの出力特性を表す行列 $\mathbf{P}_{\text{STATS}}$ を得るため、ランダムなバイナリデータを HEDGES で符号化した複数の DNA 鎖に対し、ナノポアシーケンサーによる電流波形を Squigulator を用いてシミュレーションした。次に、シミュレーションされた電流波形に対して Bonito を用いて CTC 出力を取得し、3.1.2 節で述べた方法で $\mathbf{P}_{\text{CTC}}$ を得た。ここで、HEDGES 符号化結果における j 番目の DNA 鎖の i 番目の塩基を $C_i^{(j)}$ と表し、この塩基配列を $\{C_i^{(j)}\}$ とする。また、 $\{C_i^{(j)}\}$ のベースコールにおいて得られた $\mathbf{P}_{\text{CTC}}$ を $\mathbf{P}_{\text{CTC}}^{(j)}$ と表すこととし、 $\mathbf{P}_{\text{CTC}}^{(j)}$ の各行において、出力確率が最大となる塩基を選択することで得られる塩基配列を $\{C_i'^{(j)}\}$ と表す。この $\{C_i'^{(j)}\}$ と $\{C_i^{(j)}\}$ に対してグローバルアラインメントを行い、各塩基ごとに正しい読み出し、置換・挿入・消失エラーの発生数を集計した [16]。これを全ての j について行い、集計することで各事象の発生確率を計算し、行列 $\mathbf{P}_{\text{STATS}}$ を得た。

$\mu_{\text{STATS_ok}}$, $\mu_{\text{STATS_sub}}$, $\mu_{\text{STATS_ins}}$, $\mu_{\text{STATS_del}}$ はそれぞれ、正しい読み出し、置換エラー、挿入エラー、消失エラーが発生したときの $\mathbf{P}_{\text{STATS}}$ のスコアの期待値であ

るから,

$$\mu_{\text{STATS_ok}} = \frac{\sum_{X \in \{A, C, G, T\}} p_{XX} \mathbf{P}_{\text{STATS}}(X, X)}{\sum_{X \in \{A, C, G, T\}} p_{XX}} \quad (\text{A.1})$$

$$\mu_{\text{STATS_sub}} = \frac{\sum_{X, Y \in \{A, C, G, T\}, X \neq Y} p_{XY} \mathbf{P}_{\text{STATS}}(X, Y)}{\sum_{X, Y \in \{A, C, G, T\}, X \neq Y} p_{XY}} \quad (\text{A.2})$$

$$\mu_{\text{STATS_ins}} = \frac{\sum_{Y \in \{A, C, G, T\}} p_{NY} \mathbf{P}_{\text{STATS}}(N, Y)}{\sum_{Y \in \{A, C, G, T\}} p_{NY}} \quad (\text{A.3})$$

$$\mu_{\text{STATS_del}} = \frac{\sum_{X \in \{A, C, G, T\}} p_{XN} \mathbf{P}_{\text{STATS}}(X, N)}{\sum_{X \in \{A, C, G, T\}} p_{XN}} \quad (\text{A.4})$$

により計算される． $\mu_{\text{CTC_ok}}$, $\mu_{\text{CTC_sub}}$, $\mu_{\text{CTC_ins}}$ はそれぞれ, $\mathbf{P}_{\text{CTC}}$ において正しい読み出し, 置換エラー, 挿入エラーに対応するスコアの期待値であるから,

$$\mu_{\text{CTC_ok}} = \frac{1}{M} \sum_j \sum_i \mathbf{P}_{\text{CTC}}^{(j)}(i, C_i^{(j)}) \quad (\text{A.5})$$

$$\mu_{\text{CTC_sub}} = \frac{1}{3M} \sum_j \sum_i \sum_{X \in \{A, C, G, T\}, X \neq C_i^{(j)}} \mathbf{P}_{\text{CTC}}^{(j)}(i, X) \quad (\text{A.6})$$

$$\mu_{\text{CTC_ins}} = \frac{1}{M} \sum_j \sum_i \mathbf{P}_{\text{CTC}}^{(j)}(i, N) \quad (\text{A.7})$$

により計算される．ただし, M はベースコールされた全ての DNA 鎖における塩基数の総和を表す．

次に, 式 (3.2) から (3.5) に示した固定の報酬とペナルティ P_{ok} , P_{sub} , P_{ins} , P_{del} 及び各補正項の重み付けパラメータ α_{ok} , α_{sub} , α_{ins} , α_{del} , β_{ok} , β_{sub} , β_{ins} の最適な値を決定する．ここで, P_{ok} には文献 [10] 用いられた値を各符号化率ごとに使用し, P_{ok} 以外のパラメータを最適化した．各パラメータの最適化は, 復号におけるヒープの探索数を目的関数として数値微分を用いた勾配降下法により行った．具体的には, まずランダムに生成したバイナリデータを HEDGES で符号化し, ナノポアシー

センサーでの読み出しをシミュレーションした上でその電流波形に対してベースコーラーを適用し、CTC 出力を得た。勾配降下法によるパラメータの最適化は、この CTC 出力の復号を対象として、以下の手順で行った。

1. 最適化対象の各パラメータを適当な初期値で初期化する。
2. 提案手法を用いて復号を実施し、復号が完了するまでのヒープの探索数を計測する。
3. 各パラメータごとに微小な値 Δ を加えた場合の復号におけるヒープの探索数を計測する。
4. これらの値を用いて各パラメータに関する目的関数の勾配を計算する。
5. 各パラメータを学習率 η を用いて以下のように更新する。

$$\theta \leftarrow \theta - \eta \frac{\partial J}{\partial \theta} \quad (\text{A.8})$$

ここで、 θ は最適化対象の各パラメータを表し、 J は目的関数を表す。

6. 2 から 5 を一定回数繰り返す。

パラメータの最適化については、ナノポアシーセンサーの読み出し特性に基づく補正項と CTC 出力に基づく補正項の両方を用いる場合と、それぞれ一方のみを用いる場合について効果を検証するため、 $P_{\text{sub}}, P_{\text{ins}}, P_{\text{del}}, \alpha_{\text{ok}}, \alpha_{\text{sub}}, \alpha_{\text{ins}}, \alpha_{\text{del}}, \beta_{\text{ok}}, \beta_{\text{sub}}, \beta_{\text{ins}}$ の 10 個のパラメータに対して行うのに加えて、 $\alpha_{\text{ok}} = \alpha_{\text{sub}} = \alpha_{\text{ins}} = \alpha_{\text{del}} = 0$ として他の 6 個のパラメータに対してのみ行う場合と $\beta_{\text{ok}} = \beta_{\text{sub}} = \beta_{\text{ins}} = 0$ として他の 7 個のパラメータに対してのみ行う場合についても実施した。なお、これらの最適化は符号化率 0.75, 0.60, 0.50, 0.33 についてそれぞれ独立に行った。

A.2 実験条件

行列 $\mathbf{P}_{\text{STATS}}$ を得るためのシミュレーションでは、ランダムに生成した 400 KB のバイナリデータを HEDGES で符号化率を 0.50 として符号化した。各 DNA 鎖の塩基長は 400 nt とし、合計 12240 本の DNA 鎖を生成した。Squigulator による電流値シミュレーションのパラメータ設定、ベースコーラーのモデルについては 4.1 節の実験と同様である。

ベースコーラーのファインチューニングには、ランダムに生成した 200 KB のバイナリデータを HEDGES で符号化率 0.50, 塩基長 400 nt で符号化して得られた 6120 本の DNA 鎖と、これらの DNA 鎖に対して Squigulator でシミュレーションした電流波形を学習データとして用いた。既存の学習済みモデルである dna_r9.4.1@v2 の重みを初期値として、バッチサイズを 64 として 1 エポックあたり 75 ステップの学習を行い、エポック数は 50 としてファインチューニングを行った。

$\{C_i^{(n)}\}$ と $\{C_i'^{(n)}\}$ に対するアラインメントには Biopython の Gotoh アルゴリズムを用いたグローバルアラインメントを使用した [19, 16]。

パラメータの最適化では、ランダムに生成した 400 KB のバイナリデータを符号化率 0.75, 0.60, 0.50, 0.33 で塩基長を 400 nt として HEDGES 符号化したものに対して電流波形シミュレーションとベースコールを行って得られた CTC 出力を用いた。 P_{ok} は各符号化率ごとに表 A.1 に示す固定値を使用した。各パラメータの初期値は $P_{sub} = P_{ins} = P_{del} = 1.0$, $\alpha_{ok} = \alpha_{sub} = \alpha_{ins} = \alpha_{del} = 0.0$, $\beta_{ok} = \beta_{sub} = \beta_{ins} = 0.0$ とした。学習率は符号化率ごとに表 A.2 に示す値を使用し、パラメータの更新を 300 回繰り返した。なお、 $\mathbf{P}_{STATS}$ を得るためのシミュレーション、ベースコーラーのファインチューニング、パラメータの最適化に用いたランダムなバイナリデータは全て独立に生成したものである。

A.3 結果

ナノポアシーケンサーの読み出し特性の評価により得られたラベル $\{N, A, C, G, T\}$ 間の遷移確率は表 A.2 に示す。この結果では、正しい読み取りは全体のうち 99.1% を占めており、置換エラーは 0.421%, 挿入エラーは 0.133%, 消失エラーは 0.302% であった。また、置換エラーは A,G 間と C,T 間で発生しやすい傾向が見られ、特に、最も確率の低い A,T 間の置換と比べて A,G 間の置換は約 23 倍の確率で発生することが分かった。塩基ごとの置換エラー発生率の偏りは、プリン塩基 (A,G) とピリミジン塩基 (C,T) の化学構造の類似性に起因すると考えられている [20]。N から A,C,G,T への遷移で表わされる挿入エラーは、各塩基に対して同程度の確率で発生していた。A,C,G,T から N への遷移で表わされる消失エラーは、A,G において C,T よりも比較的高い確率で発生していた。この結果から得られた行列 $\mathbf{P}_{STATS}$ は式 (A.9) のようになった。各補正項の期待値は表 A.3 に示す。各符号化率ごとに最適化されたペナルティと補正項の重み付けパラメータは表 A.4 お

よび表 A.5 に示す.

$$\boldsymbol{P}_{\text{STATS}} = \begin{bmatrix} -1.20 & -0.650 & -0.650 & -0.643 & -0.613 \\ -0.580 & 0.130 & -0.670 & -0.590 & -1.25 \\ -0.620 & -0.650 & 0.110 & -0.640 & -1.09 \\ -0.550 & -0.520 & -0.590 & 0.160 & -1.53 \\ -0.590 & -1.53 & -1.09 & -1.25 & 0.120 \end{bmatrix} \quad (\text{A.9})$$

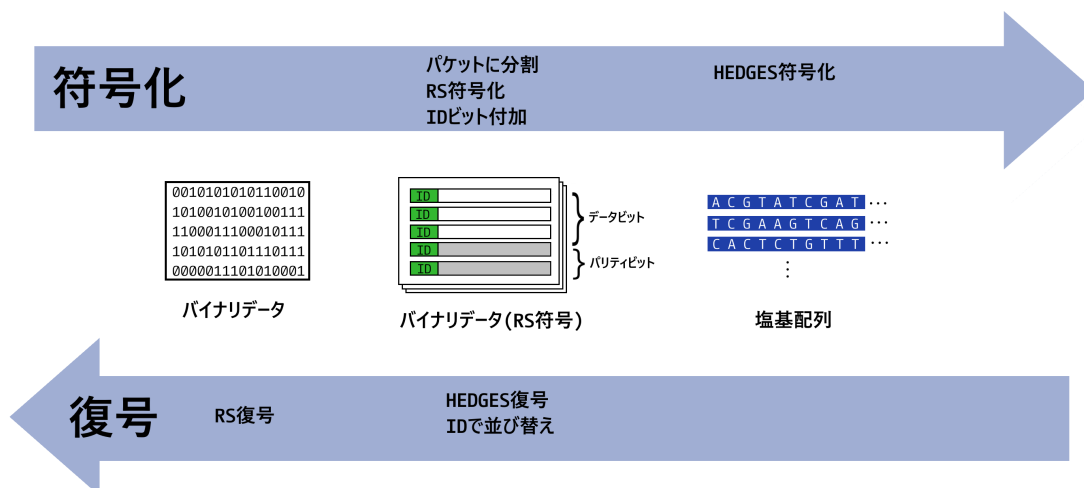


図 2.1: Overview of HEDGES encoding

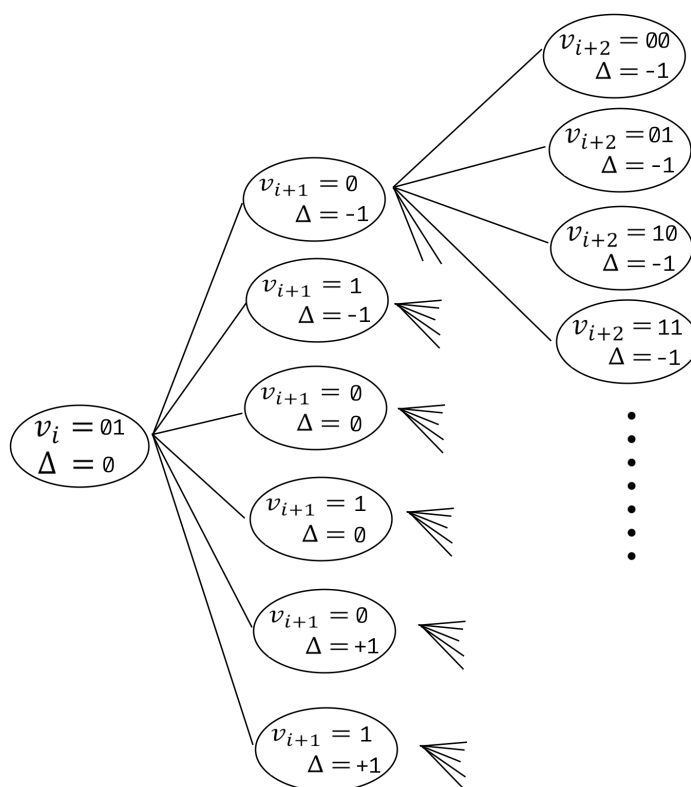


図 2.2: Heap structure in HEDGES decoding

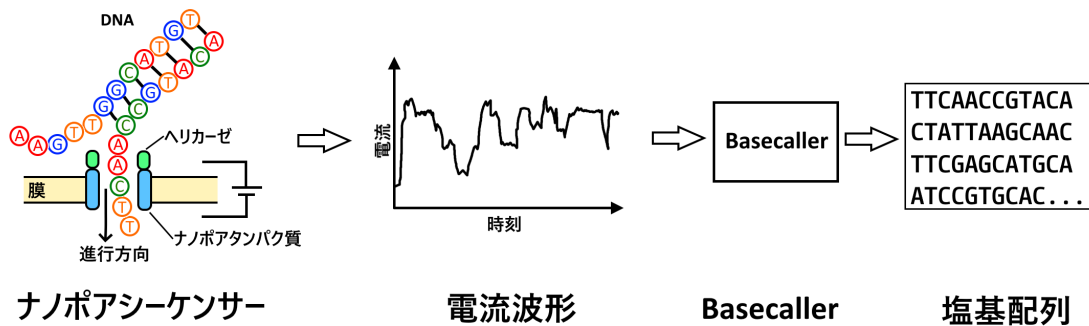


図 2.3: Principle and procedure of nanopore sequencing

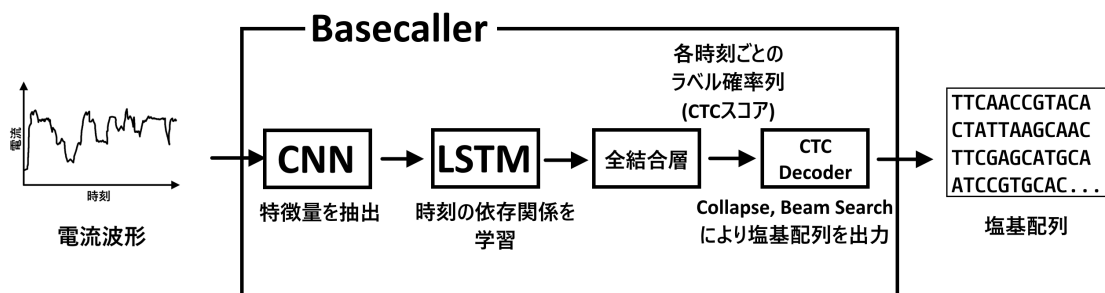


図 2.4: Overview of the Bonito architecture

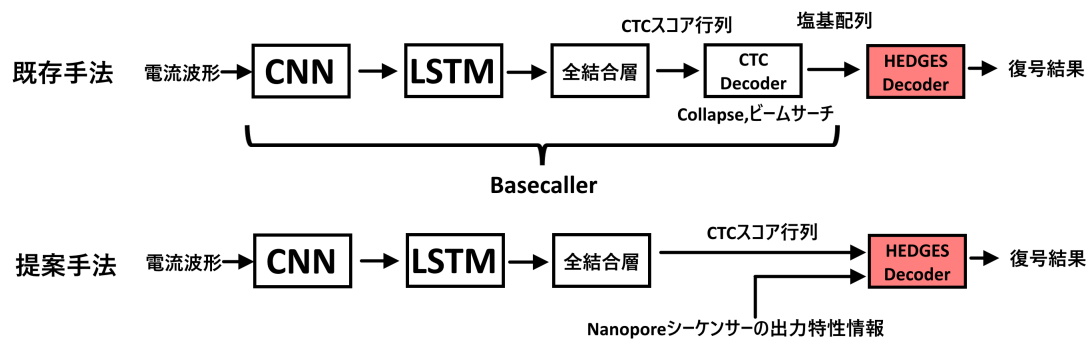


図 3.1: Overview of the proposed method

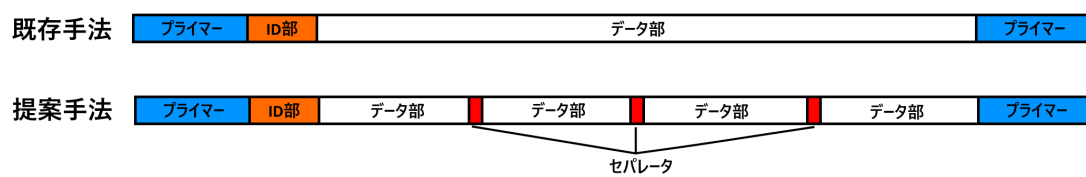


図 3.2: Encoding scheme of the proposed method

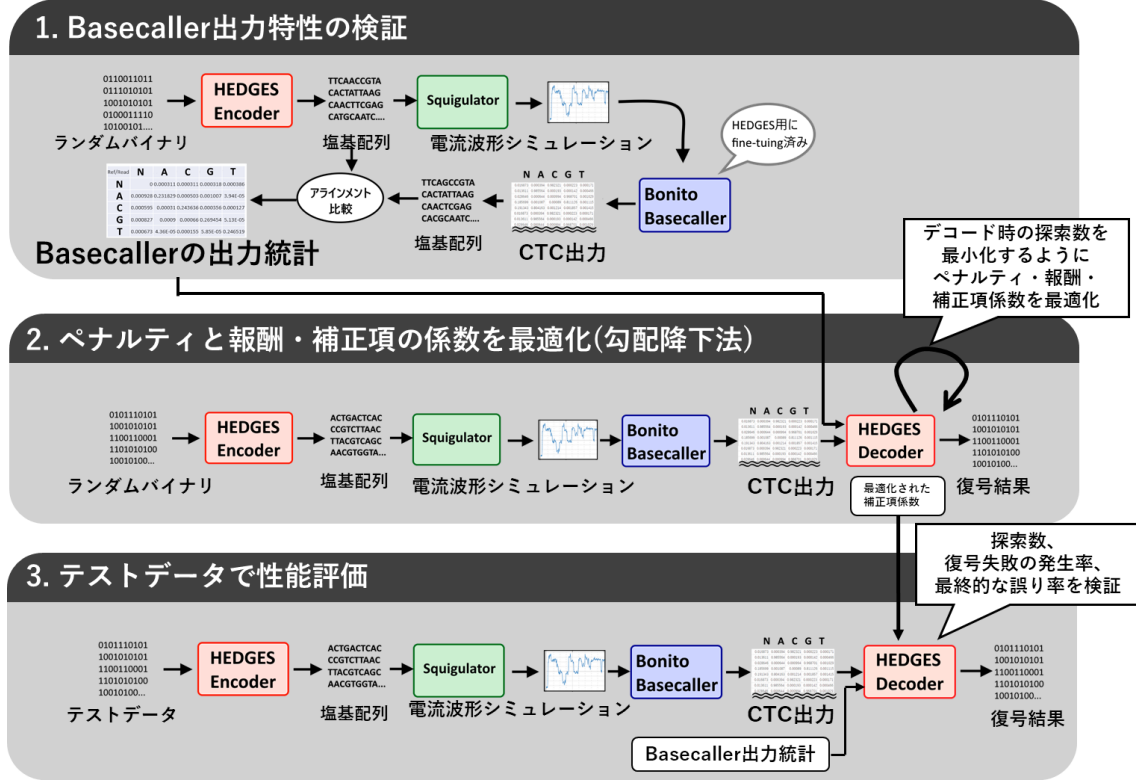


図 4.1: Overview of the simulation procedure

表 4.1: Average number of heap searches per DNA strand

Coding rate		0.75	0.6	0.5	0.33
Existing method		135550	45219	21291	12290
Proposed method	STATS only	77281	23707	10893	4785
	CTC only	53098	17966	9353	4354
	STATS+CTC	48507	14972	7625	3894

表 4.2: Bit-erasure rate

Coding rate		0.75	0.6	0.5	0.33
Existing method		0.03083	0.00949	0.00341	0.00328
Proposed method	STATS only	0.00741	0.00376	0.00121	0.00076
	CTC only	0.00789	0.00265	0.00111	0.00055
	STATS+CTC	0.00693	0.00229	0.00069	0.00056

表 4.3: Bit error rate after RS decoding

Coding rate		0.75	0.6	0.5	0.33
Existing method		0.022981	$< 10^{-6}$	$< 10^{-6}$	$< 10^{-6}$
Proposed method	STATS only	0.059977	$< 10^{-6}$	$< 10^{-6}$	$< 10^{-6}$
	CTC only	0.008228	$< 10^{-6}$	$< 10^{-6}$	$< 10^{-6}$
	STATS+CTC	0.005173	$< 10^{-6}$	$< 10^{-6}$	$< 10^{-6}$

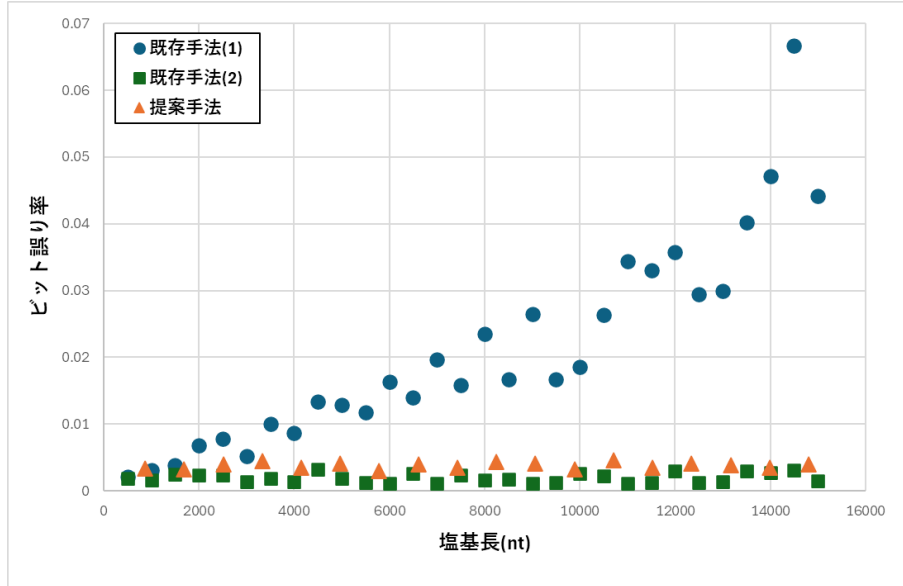


図 4.2: Relationship between base length and bit error rate

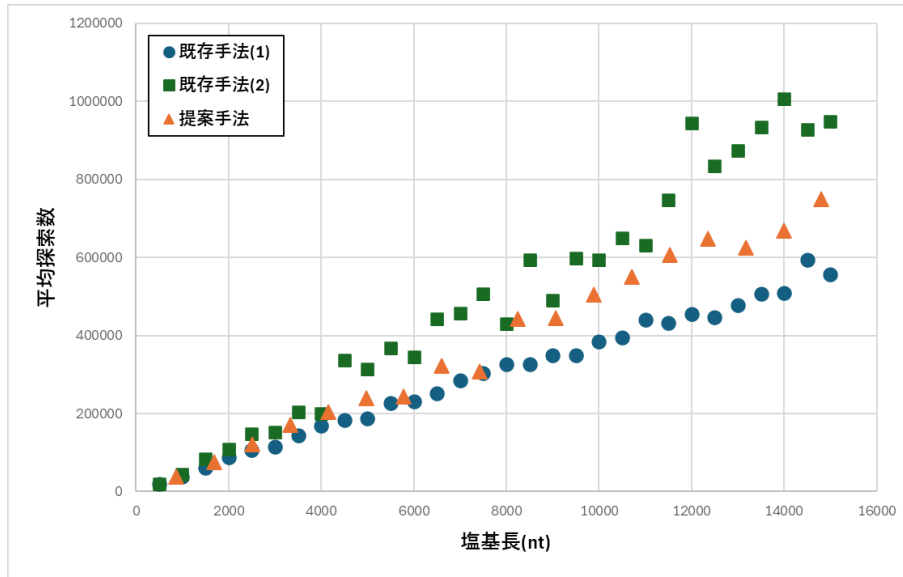


図 4.3: Relationship between base length and average number of heap searches per DNA strand

表 A.1: 符号化率ごとの報酬値 P_{ok}

符号化率	P_{ok}
0.75	-0.035
0.60	-0.082
0.50	-0.127
0.33	-0.229

表 A.2: 符号化率ごとの学習率 η

符号化率	η
0.75	1.3×10^{-7}
0.60	4.1×10^{-7}
0.50	1.0×10^{-6}
0.33	3.8×10^{-6}

表 A.2: ラベル間の遷移確率

入力 \ 出力	N	A	C	G	T
N	0.000	3.11×10^{-4}	3.11×10^{-4}	3.18×10^{-4}	3.86×10^{-4}
A	9.28×10^{-4}	0.232	5.03×10^{-4}	1.01×10^{-3}	3.94×10^{-5}
C	5.95×10^{-4}	3.10×10^{-4}	0.244	3.56×10^{-4}	1.27×10^{-4}
G	8.27×10^{-4}	9.00×10^{-4}	6.60×10^{-4}	0.269	5.13×10^{-5}
T	6.73×10^{-4}	4.36×10^{-5}	1.55×10^{-4}	5.85×10^{-5}	0.247

表 A.3: 得られた各期待値

パラメータ	値
$\mu_{\text{STATS_ok}}$	1.38
$\mu_{\text{STATS_sub}}$	7.52
$\mu_{\text{STATS_ins}}$	8.01
$\mu_{\text{STATS_del}}$	7.17
$\mu_{\text{CTC_ok}}$	0.111
$\mu_{\text{CTC_sub}}$	6.85
$\mu_{\text{CTC_ins}}$	2.78

表 A.4: 符号化率ごとに最適化したペナルティ

符号化率	手法	P_{sub}	P_{ins}	P_{del}
0.75	STATS のみ *	0.630	0.650	0.687
	CTC のみ **	0.837	0.912	0.765
	STATS+CTC***	0.875	0.911	0.778
0.60	STATS のみ *	0.883	0.965	0.995
	CTC のみ **	0.856	1.06	0.890
	STATS+CTC***	1.08	1.17	1.02
0.50	STATS のみ *	0.988	1.17	1.25
	CTC のみ **	0.953	1.11	1.04
	STATS+CTC***	1.15	1.23	1.19
0.33	STATS のみ *	1.07	1.25	1.39
	CTC のみ **	1.06	1.22	1.21
	STATS+CTC***	1.29	1.35	1.34

*STATS のみ: ナノポアシーケンサーの読み出し特性に基づく補正項のみを使用

**CTC のみ: CTC 出力に基づく補正項のみを使用

***STATS+CTC: ナノポアシーケンサーの読み出し特性および CTC 出力の両方に基づく補正項を使用

表 A.5: 符号化率ごとに最適化した補正項の重み付けパラメータ

符号化率	手法	α_{ok}	α_{sub}	α_{del}	α_{ins}	β_{ok}	β_{sub}	β_{ins}
0.75	STATS のみ	0.0605	0.116	0.0929	0.0952	0.00	0.00	0.00
	CTC のみ	0.00	0.00	0.00	0.00	-0.192	0.211	0.347
	STATS+CTC	0.00668	0.130	-0.0105	0.544	-0.150	0.181	0.301
0.60	STATS のみ	0.00513	0.256	0.200	0.0969	0.00	0.00	0.00
	CTC のみ	0.00	0.00	0.00	0.00	-0.223	0.190	0.419
	STATS+CTC	0.110	0.243	0.236	0.219	-0.115	0.184	0.330
0.50	STATS のみ	0.0548	0.294	0.0800	0.0871	0.00	0.00	0.00
	CTC のみ	0.00	0.00	0.00	0.00	-0.108	0.193	0.411
	STATS+CTC	0.131	0.292	0.234	0.176	-0.136	0.188	0.467
0.33	STATS のみ	-0.00773	0.282	0.167	0.246	0.00	0.00	0.00
	CTC のみ	0.00	0.00	0.00	0.00	0.0272	0.145	0.422
	STATS+CTC	0.0260	0.328	0.177	0.230	0.0417	0.134	0.460